

problem 2 explanation

problem

here are what's given to us:

- N : the number of cows
- M : how many minutes we need to simulate
- s : the string of directions
- a : starting amount / total capacity of milk

we need to find the amount of milk after M minutes.

idea

the key is to see that most cows don't actually lose any milk, and we just need to find the ones that do and figure out how much they lose. then, we can subtract the amount of milk lost from the total amount of milk we have at the beginning.

first, before starting the simulation, we need to identify the cows that are in a stable position, and the cows that are destined to lose milk.

we can classify each cow into three positions:

- **sinks**: they get milk from both sides. since they can hold at most a_i litres, they will spill any of the milk that were given to them. in code, we represent it as `inflow == 2`.
- **conveyors**: they give one litre and receive one litre – until the cow that gives them runs out of milk. in code, we represent it as `inflow == 1`.
- **drains**: they get no milk, but still give away each minute; losing milk at the start. in code, we can represent it as `inflow == 0`.

when a cow becomes a **drain**, it stops passing milk to its neighbour, causing a chain reaction. if the neighbour cow was a **conveyor**, it will start losing milk as well.

algorithm

our strategy is to identify when these chains happen and calculate the amount of milk lost. for efficiency, we can use an event-based simulation and only jump to the moments when a cow runs out of milk, skipping the unneeded calculations.

1. **calculate state**: we loop through all N cows, and calculate the inflow for each cow to see if it's a **sink**, **conveyor**, or **drain**. we also figure out which cow each cow i passes its milk to and store it in a **target** array – which helps us quickly find which cow is affected later on.
2. **initial events**: we use a **priority_queue** to keep track of events; which will always give us the event that happens first. when a cow runs out of milk, we'll store it in the priority queue as a pair: `{ time_of_emptiness,`

`cow_index }`. we loop through the cows again – if a cow is a **drain** (`inflow == 0`), we know it's going to run out of milk at time $t = a_i$; and we can add this event to our queue. otherwise, if a cow is a **sink** (`inflow == 2`), we add it to a `num_sinks` counter. this counter tells us how many litres of milk we lose per minute per sink.

3. **run the simulation:** we keep running this loop as long as there are events in our queue. in the loop, we first take the event with the soonest time from our queue, which gives us a `time_of_emptiness` and `cow_index` pair.

we can call the cow at the `cow_index` cow `u`, and it will run out at time `time_of_emptiness`, which we name it `t_empty`. we then calculate the amount of milk lost, which is `num_sinks * (t_empty - current_time)`. we add this to our `total_loss`, and skip the time to when the cow runs out of milk.

after that, we need to continue the chain reaction. since cow `u` is empty, it will stop giving milk to its neighbour cow `v` – which we can get from the `target` array we calculated earlier. we then decrease `inflow[v]`, which can turn a **sink** to a **conveyor**, or a **conveyor** to a **drain****, which can cause a chain reaction again.

if **sink** $\rightarrow$ **conveyor**, we decrease `num_sinks`. if **conveyor** $\rightarrow$ **drain**, we add a new event to the queue for when cow `v` will run out of milk.

4. **final calculation:** after the loop finishes – either because the queue is empty or the next event is after time M – we calculate the loss for any remaining time up to M .
5. **answer:** the final amount of milk is just the `total_initial_milk - total_loss`.